FinalProjectReport.docx

AUT University

Document Details

Submission ID
trn:oid:::20437:140402168

Submission Date
May 26, 2026, 1:48 AM GMT+12

Download Date
May 26, 2026, 2:00 AM GMT+12

File Name
FinalProjectReport.docx

File Size
34.6 KB

14 Pages

4,804 Words

30,450 Characters

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **25** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
5% Submitted works (Student Papers)

Match Groups

- **25** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
5% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Bahrain Polytechnic on 2026-05-16 — 1%
2	Submitted works	AUT University on 2026-04-19 — <1%
3	Submitted works	Brunel University on 2025-04-16 — <1%
4	Submitted works	Islington College,Nepal on 2026-04-29 — <1%
5	Submitted works	University of Wales Institute, Cardiff on 2025-12-23 — <1%
6	Submitted works	University of Canberra on 2024-01-17 — <1%
7	Submitted works	QA Learning on 2024-03-22 — <1%
8	Submitted works	National University on 2026-05-18 — <1%
9	Submitted works	Murdoch University on 2025-10-13 — <1%
10	Submitted works	University of Cape Town on 2026-04-30 — <1%

11 Submitted works

Asia Pacific Instutute of Information Technology on 2025-02-01 <1%

12 Submitted works

Regent Independent School and Sixth Form College on 2025-09-20 <1%

13 Submitted works

University of Wales Institute, Cardiff on 2026-03-07 <1%

14 Submitted works

Asia Pacific University College of Technology and Innovation (UCTI) on 2021-11-30 <1%

2

Data Process & Software Modelling

Assessment Phase 2: Final Project Progress Report and Presentation (50%)

Presented By:
Christian Danielle B. Cantos

Submitted To:
Abubakar Siddique

Github repository: https://github.com/Ianieee36/ENSE706_Project.git

May 2026
Task 5: Refined Static and Dynamic Modelling

Refined UML Class Diagram with Abstraction and interfaces

14

The refined UML class diagram significantly improves the structure of the Flight Booking System through the introduction of a layered architecture and stronger object-oriented design principles. The system is organised into four layers: Model, Repository, Service, and UI, allowing responsibilities to be clearly separated between domain modelling, data access, business logic, and user interaction. This refinement reduces coupling between components and creates a more maintainable and extensible system architecture.

The Model layer was enhanced by restructuring the user hierarchy using inheritance. Instead of relying solely on a role enumeration within a single User class, the refined design introduces Customer and Admin subclasses, with User acting as an abstract superclass containing shared user attributes and behaviours. This provides a more accurate object-oriented representation while allowing specialised functionality to be encapsulated within each subclass.

The refined design also introduces the Ticket entity, enabling ticket management to be handled independently from booking operations. This improves the completeness of the domain model and better reflects real-world airline reservation systems. Additionally, the system applies the Repository Pattern and Service Layer Pattern through dedicated repository and service classes, ensuring clear separation between persistence logic and business logic.

Furthermore, interface-based abstractions such as IUserRepository, IFlightRepository, IBookingRepository, and their corresponding service interfaces improve flexibility and support future extensibility. Overall,

the refined UML class diagram demonstrates a transition from a basic conceptual design to a more structured and scalable software architecture aligned with object-oriented and layered design principles.

Minimum of three Sequence Diagrams

User Login

The login sequence diagram illustrates the interaction between the user interface, service layer, repository layer, and database during the user authentication process. The sequence begins when the user enters their login details, specifically the email and password, through the MainMenu. The MainMenu then sends a Login(email, passwordHash) request to the UserService, which is responsible for handling the business logic related to authentication.
The UserService communicates with the UserRepository by calling FindUserByEmail(email) to retrieve the user information from the database. The UserRepository executes an SQL query to search for a matching user record in the USERS table and returns the corresponding user object to the UserService. Depending on the retrieved user type, the system identifies whether the authenticated user is a Customer or an Admin, reflecting the inheritance-based design where both entities inherit from the abstract User class.
An alt combined fragment is used to represent the different possible outcomes of the login process. If the credentials belong to a customer, the system returns a Customer object and displays the customer menu. If the credentials belong to an administrator, the system returns an Admin object and displays the administrator menu. Otherwise, if the login credentials are invalid, the authentication fails and an error message is displayed to the user. This sequence diagram demonstrates how the layered architecture separates presentation logic, business logic, and data access responsibilities while supporting polymorphism through inheritance.

Booking Flight

The Book Flight sequence diagram illustrates the interaction between the

customer, user interface, service layer, repository layer, and database during the flight booking process. The sequence begins when the customer selects the booking option through the CustomerMenu, which sends a BookFlight(customer, flightId) request to the BookService. The BookService first communicates with the FlightRepository to retrieve the selected flight using FindFlightById(flightId). The FlightRepository queries the database and returns the corresponding Flight object to the service layer.
After retrieving the flight information, the BookService communicates with the BookingRepository to obtain all existing bookings associated with the customer by calling FindBookingsByUserId(userId). The returned list of bookings is then checked to determine whether the customer has already booked the same flight and whether the booking status is not cancelled. This validation is represented using an alt combined fragment. If the customer has already booked the flight, the booking process fails and an appropriate message is displayed informing the customer that the flight has already been booked.
If the customer has not previously booked the flight, the system proceeds to check seat availability using flight.HasAvailableSeats(). A second alt fragment is used to represent the two possible outcomes. If there are no available seats, the booking request fails and the customer is notified that no seats are available. Otherwise, if seats are available, the BookService generates a unique booking identifier and creates a new booking. The booking information is then saved into the database through the BookingRepository using an INSERT INTO BOOKINGS operation.
Once the booking has been successfully stored, the system updates the flight seat count by calling flight.UpdateSeatCount(-1) to reduce the available seats by one. The updated flight information is then persisted in the database through the FlightRepository using an UPDATE FLIGHTS operation. Finally, the newly created booking is returned to the CustomerMenu, and a booking confirmation message is displayed to the customer. This sequence diagram demonstrates the complete booking workflow, including validation checks, seat management, and database persistence while maintaining separation of concerns through the layered system architecture.

Search Flight / View Flight Details

The Search Flight and View Flight Details sequence diagram shows how a customer or guest searches for available flights and optionally views more information about a selected flight. The process begins when the user sends a HandleSearchFlight(customer) request through the CustomerMenu. The menu then calls SearchFlights(origin, destination) from

the FlightService, which passes the request to the FlightRepository using FindFlightsByRoute(origin, destination). The repository queries the database for flights that match the given origin and destination, then returns a list of matching flight records back through the repository and service layer to the menu.
An alt combined fragment is used to represent the result of the search. If flights are found, the system displays the available flights to the customer or guest. The user can then choose to view more details about a specific flight. The CustomerMenu sends a GetFlightDetailsById(flightId) request to the FlightService, which retrieves the selected flight from the FlightRepository. The repository queries the database using the selected flightId and returns the matching flight record. The flight object is then returned to the menu, and the full flight details are displayed to the user.
If no matching flights are found, the alternative path is followed and the system displays a "No flights found" message. This sequence diagram demonstrates how the search and view details functions work together inside the customer menu while maintaining a clear separation between the user interface, service layer, repository layer, and database.
Minimum two Activity Diagrams and /or State Machine Diagrams

Login Activity Diagram

The Login Activity Diagram illustrates the workflow of the user authentication process within the flight booking system. The process begins when the user enters their email and password, followed by submitting the login details to the system. The system then attempts to locate the user by searching for a matching email address in the database. A decision node is used to determine whether the user exists. If no matching user record is found, the system displays a "User not found" message and the activity terminates.
If the user exists, the system proceeds to validate the entered password. Another decision node checks whether the password is valid. If the password is incorrect, the system displays an "Invalid credentials" message before ending the process. However, if the password is successfully validated, the system determines the type of authenticated user. Since the system implements inheritance-based user management, the authenticated user may either be a Customer or an Admin. Depending on the identified user type, the appropriate menu interface is displayed, either the customer menu or the administrator menu.
The activity diagram demonstrates the complete decision-making workflow of the login process, including user verification, password validation, and role-based navigation. It also clearly shows the alternative execution paths and termination points, providing a visual representation of the authentication logic implemented within the system.

Book Flight Activity Diagram

The Book Flight Activity Diagram illustrates the workflow involved when a customer attempts to book a flight within the flight booking system. The process begins when the customer selects a flight to book, after which the system retrieves the flight information using the provided flightId. A decision node is then used to determine whether the selected flight exists. If the flight cannot be found, the system displays a "Flight not

found" message and the activity terminates.
If the flight exists, the system retrieves all existing bookings associated with the customer in order to validate whether the customer has already booked the same flight and that the booking has not been cancelled. Another decision node is used to evaluate this condition. If the customer has already booked the flight, the system displays an "Already booked" message and ends the booking process.
If the customer has not previously booked the selected flight, the system proceeds to check seat availability. A further decision node determines whether seats are still available for the flight. If no seats are available, the system displays a "No seats available" message and terminates the activity. However, if seats are available, the booking process continues by generating a unique booking identifier, creating a new booking object, and saving the booking information into the database. The system then updates the available seat count and persists the updated flight record. Finally, a booking confirmation message is displayed to the customer, indicating that the booking was successfully completed.
This activity diagram demonstrates the complete booking workflow, including validation checks, decision-making paths, booking creation, and seat management. It also highlights how alternative outcomes such as duplicate bookings, unavailable seats, and missing flights are handled within the system.

State Machine Diagram

Booking State

The Booking State Machine Diagram illustrates the lifecycle of a booking object within the flight booking system. The diagram begins at the initial state, where a booking is created through the BookFlight() operation. Once the booking process is successfully completed, the booking transitions into the CONFIRMED state, indicating that the customer has successfully reserved a seat on the selected flight and the booking has been stored in the system.
From the CONFIRMED state, the customer may choose to cancel the booking by triggering the CancelBooking() operation. This causes the booking to transition into the CANCELLED state, representing that the reservation is no longer active. Once the booking reaches the CANCELLED state, the lifecycle of the booking ends and transitions to the final state of the diagram.
This state machine diagram provides a simplified representation of the booking lifecycle implemented in the system. It demonstrates how a booking changes state in response to specific operations and reflects the current implementation of the booking process, where bookings are directly confirmed upon successful creation and may later be cancelled by the customer.

Task 6: Design Pattern Application and Implementation

Application of at least two GoF design patterns

The refined Flight Booking System applies two GoF design patterns: the Singleton Pattern and the Factory Method Pattern. These patterns were selected because they directly address two important design concerns in the system: centralised database connection management and controlled creation of specialised user objects.

13 The Singleton Pattern is implemented in the DatabaseConnection class. This ensures that the repository layer uses one shared database connection manager instead of creating separate connection configuration objects across different repositories. Since UserRepository, FlightRepository, BookingRepository, and TicketRepository all require database access, the Singleton provides a consistent global access point through DatabaseConnection.Instance. This reduces duplicated connection setup code and keeps database configuration in one dedicated class.

The Factory Method Pattern is implemented through the UserFactory class.

4 Since the system now uses an inheritance-based user model, where User is the superclass and Customer and Admin are specialised subclasses, object creation is handled through factory methods rather than scattered constructor calls. The CreateCustomer() method creates customer objects with attributes such as loyaltyPoints and membershipTier, while CreateAdmin() creates administrator objects with an adminLevel. This keeps user creation logic separate from the service and repository layers.

Together, these patterns improve the organisation of the system by assigning database connection management and object creation to dedicated classes. This results in clearer responsibility allocation, less duplicated logic, and a design that is easier to extend when new repositories, user types, or database configuration changes are introduced.

UML diagrams showing pattern structure

UserFactory Class

The UML pattern diagrams provide a structural view of how the Singleton and Factory Method patterns are applied within the system. Rather than focusing on the general purpose of each pattern, these diagrams show the specific classes, relationships, and responsibilities involved in the implementation.
The Factory Method structure is represented through the UserFactory, User, Customer, and Admin classes. The diagram shows Customer and Admin inheriting from User, while UserFactory has dependency relationships with both subclasses because it is responsible for creating their objects. This demonstrates how object creation is separated from the main business logic while still supporting the inheritance-based user model.

DatabaseConnection Class

The Singleton structure is represented through the DatabaseConnection class and its relationship with the repository classes. The diagram highlights the private constructor, static instance field, and public Instance property, which together enforce controlled access to a single shared connection manager. The repository classes depend on DatabaseConnection to obtain database connections, showing how connection access is centralised within the repository layer.
These UML diagrams support the implementation by making the design pattern structures explicit and showing how each pattern is integrated into the overall class architecture.

Corresponding C# implementation demonstrating each pattern

Singleton Pattern in DatabaseConnection Class

4 The DatabaseConnection class demonstrates the implementation of the Singleton Design Pattern by ensuring that only one shared instance of the database connection manager exists throughout the system. This is
8 achieved through the use of a private static variable named instance, which stores the single object of the class. The constructor is declared as private, preventing external classes from directly creating new DatabaseConnection objects. Instead, the system accesses the shared object through the public static Instance property, which checks whether an instance already exists and creates it only if necessary. This

mechanism guarantees controlled global access to a single shared connection manager.
Additionally, the GetConnection() method is responsible for creating and returning an OracleConnection object using the configured connection string, while the TestConnection() method verifies that the database connection is functioning correctly. By implementing the Singleton Pattern in this class, the system centralises database connection configuration, reduces unnecessary object creation, and allows all repository classes to consistently access the same database connection manager.

Factory Method (UserFactory)

The UserFactory class demonstrates the implementation of the Factory Method Design Pattern by centralising the creation of specialised user objects within the flight booking system. Instead of directly instantiating Customer and Admin objects throughout the application using constructors, the system delegates object creation responsibilities to the factory methods CreateCustomer() and CreateAdmin(). Each factory method accepts the required user information as parameters and returns a fully initialised object of the appropriate subclass.
The CreateCustomer() method creates and returns a Customer object, including customer-specific attributes such as loyaltyPoints and membershipTier, while the CreateAdmin() method creates and returns an Admin object with the adminLevel attribute. This implementation encapsulates the object creation logic within a dedicated factory class, reducing duplication and improving code organisation across the system. By applying the Factory Method Pattern, the system achieves better separation of concerns, as repository and service classes no longer need to directly manage subclass instantiation logic. The pattern also improves maintainability and extensibility by allowing new user types to be added in the future with minimal modifications to the existing codebase.

Task 7: Comparative Design Analysis (Before vs After Patterns)
Initial Class Diagram

Refined Class Diagram

Changes to class structure and responsibility allocation
The initial Flight Booking System design followed a basic layered architecture consisting of the UI, service, repository, and model layers. However, responsibilities within the model layer were less specialised, particularly in the User class, which relied on a Role enumeration to distinguish between customers, administrators, and guests.
This approach resulted in repeated role-checking logic across the service and UI layers, limiting the use of inheritance and polymorphism. In addition, database connection handling was not centralised, causing repository classes to manage persistence operations without a dedicated reusable connection manager.
Following the refinement of the system architecture and the introduction of design patterns, the class structure became more modular and aligned with object-oriented principles. The updated model layer introduces inheritance through the Customer and Admin subclasses, with User acting as an abstract superclass containing shared attributes and behaviour. This allows specialised properties and operations to be encapsulated within their respective subclasses, improving the clarity of the domain model while reducing reliance on conditional role-based logic.
Responsibility allocation was further improved through the introduction of the UserFactory class, which centralises the creation of specialised user objects. Instead of distributing object instantiation logic throughout the system, user creation is now handled through dedicated factory methods, allowing service and repository classes to remain focused on business and persistence operations.
The repository layer was also refined through the implementation of the Singleton Pattern in the DatabaseConnection class. Repository classes such as UserRepository, FlightRepository, BookingRepository, and TicketRepository now share a centralised database connection manager through DatabaseConnection.Instance, eliminating duplicated connection configuration logic and improving consistency across the persistence layer.
Overall, the refined design demonstrates clearer separation of

responsibilities across architectural layers. Model classes encapsulate domain behaviour and state, repositories manage persistence operations, services coordinate business logic and validation, while dedicated factory and singleton classes handle object creation and shared resource management. This results in a more structured and maintainable system architecture compared to the original design.

Improvements in cohesion and reductions in coupling

The refined system architecture demonstrates improved cohesion by assigning more focused responsibilities to individual classes and layers. In the initial design, several components handled multiple concerns simultaneously, particularly within the user management and repository layers. The User class combined all user-related behaviour into a single general entity, while repository classes operated without a dedicated connection management component. This resulted in broader class responsibilities and tighter dependencies between system components.

The updated design introduces stronger object-oriented separation through inheritance and specialised supporting classes. The Customer and Admin subclasses now encapsulate role-specific behaviour and attributes, allowing the abstract User class to focus only on shared user functionality. This improves cohesion within the model layer because each class now represents a more clearly defined responsibility within the domain.

Coupling was also reduced through the introduction of the UserFactory and DatabaseConnection classes. User creation logic is now isolated within the factory rather than being distributed across repositories or services, reducing direct dependency on concrete subclass constructors. Similarly, repository classes no longer manage database connection configuration independently, as all persistence operations rely on the shared DatabaseConnection.Instance. This centralisation reduces duplicated logic and limits unnecessary dependencies between architectural layers.

The refined architecture also benefits from interface-based abstraction within the repository and service layers. By depending on interfaces such as IUserRepository and IBookingService rather than concrete implementations, higher-level components become less dependent on specific classes. This creates a more flexible and loosely coupled design where implementations can be modified or extended with minimal impact on the rest of the system.

Overall, the updated design achieves clearer responsibility allocation, stronger class cohesion, and lower interdependency between components, resulting in a more structured and adaptable software architecture.

Impact on extensibility and maintainability

The refined system architecture improves extensibility by making it easier to introduce new functionality without heavily modifying existing components. In the original design, expanding the system often required additional role-based conditions across multiple layers. In the updated design, new user types can be introduced by extending the abstract User

class and adding corresponding creation logic within UserFactory, allowing the system to evolve with less impact on existing functionality. Maintainability is also improved through the centralisation of key responsibilities. Database configuration is now managed within the DatabaseConnection class, meaning connection-related changes only need to be applied in one location rather than across multiple repositories. Similarly, object creation logic is encapsulated within the factory layer instead of being distributed throughout the application, reducing repeated constructor usage and simplifying future modifications to user creation processes.

T he refined architecture further supports long-term adaptability through its layered and interface-based structure. Repository classes can reuse the shared database connection manager, while service and repository interfaces allow implementations to be modified or extended independently from higher-level components. As a result, enhancements such as additional user roles, new booking features, or expanded database functionality can be integrated with minimal disruption to the existing system architecture.

Consequences for C# implementation complexity

Applying the Singleton and Factory Method patterns increased the C# implementation complexity slightly because additional classes, methods, and structural rules were introduced. For example, the DatabaseConnection class now requires a private constructor, a static instance field, and a public static Instance property to ensure that only one shared connection manager is created. This is more complex than simply creating a new DatabaseConnection object directly in each repository, but it provides better control and consistency across the repository layer.

The Factory Method Pattern also adds some implementation complexity because user object creation is no longer handled directly with simple constructor calls throughout the system. Instead, the system now uses the UserFactory class with methods such as CreateCustomer() and CreateAdmin(). This requires additional code and careful parameter handling, especially because Customer and Admin have different attributes such as loyaltyPoints, membershipTier, and adminLevel. However, this complexity is justified because it keeps object creation logic centralised and prevents duplication across repositories and services.

The refined inheritance-based design also requires more careful C# implementation than the original role-based approach. The original design could use a single User class with a Role enumeration, whereas the updated design uses an abstract User superclass with separate Customer and Admin subclasses. This means constructors must use base(...) to initialise shared user attributes, and the system must correctly decide which subclass to create. Although this adds more structure, it results in clearer and more object-oriented code.

Overall, the updated implementation is slightly more complex than the original design, but the complexity is purposeful. The additional classes and pattern-related code improve separation of concerns, reduce repeated logic, and make the system easier to modify in the future. The trade-off is that the codebase requires more careful organisation, but it provides a stronger foundation for long-term maintainability.

Examples of how new requirements can be accommodated more easily

The refined design of the Flight Booking System allows new requirements

to be integrated more easily due to the application of the Singleton and Factory Method design patterns, as well as the improved object-oriented structure. For example, if a new user type such as Staff or TravelAgent needs to be introduced in the future, the system can be extended by simply creating a new subclass that inherits from the abstract User class and adding a corresponding factory method inside the UserFactory class. Since the system already uses inheritance and centralised object creation, the addition of new user types would require minimal modification to existing service and repository classes.

Similarly, the Singleton-based DatabaseConnection class simplifies future database-related changes. If the system later migrates to a different Oracle server configuration or requires additional connection settings, the changes only need to be applied within the DatabaseConnection class. All repository classes automatically continue using the updated configuration through DatabaseConnection.Instance, eliminating the need to modify individual repositories.

The updated design also supports easier feature expansion within existing entities. For example, additional customer features such as reward redemption, VIP membership benefits, or loyalty point calculations can be implemented directly within the Customer subclass without affecting administrator functionality. Likewise, new administrative permissions can be added to the Admin class through the existing adminLevel structure. This separation of responsibilities allows specialised behaviour to evolve independently while preserving the stability of the overall system architecture.

Furthermore, the repository and service layer separation improves adaptability when implementing additional system functionality. For instance, introducing ticket validation, booking history reports, or flight scheduling enhancements can be achieved by extending the relevant service and repository classes without heavily impacting the UI or model layers. Because responsibilities are now distributed more clearly across the architecture, future modifications are more localised, reducing the likelihood of widespread code changes and making the system easier to evolve over time.

Task 8: Critical Reflection

Design trade-offs introduced by design patterns

The introduction of the Singleton and Factory Method design patterns significantly improved the structure and maintainability of the Flight Booking System; however, these refinements also introduced several design trade-offs. While the updated architecture is more modular and aligned with object-oriented design principles, it also increases the complexity of the system compared to the original implementation.

One major trade-off introduced by the Singleton Pattern is the increased

dependency on a globally accessible shared object. The DatabaseConnection class now centralises database connection management through DatabaseConnection.Instance, which improves consistency and avoids repeated connection configuration. However, this also creates a global dependency that can make unit testing and future dependency injection more difficult. In smaller systems, directly creating database connection objects may have been simpler and easier to understand, whereas the Singleton implementation introduces additional structural complexity such as static instance management and restricted constructors.

The Factory Method Pattern also introduced a trade-off between flexibility and implementation simplicity. The addition of the UserFactory class improves extensibility by centralising the creation of Customer and Admin objects, but it also increases the number of classes and abstraction layers within the system. The original role-based implementation using a single User class and a Role enumeration was simpler to implement because user objects could be instantiated directly with fewer constructors and less inheritance logic. In contrast, the refined inheritance-based design requires abstract classes, subclass constructors, factory methods, and polymorphic object handling, which increases the complexity of the C# implementation.

Another trade-off relates to scalability versus simplicity. The refined design is considerably more scalable and maintainable for future expansion, particularly when introducing additional user types or specialised behaviours. However, for a relatively small-scale academic system, some aspects of the architecture may be more sophisticated than strictly necessary. The use of multiple layers, design patterns, and inheritance structures increases the amount of code and UML modelling required, which can make the system more difficult for new developers to understand initially.

Despite these trade-offs, the introduction of design patterns ultimately provides long-term architectural benefits. The refined design promotes better separation of concerns, reduced duplication, improved extensibility, and stronger maintainability. A Although the implementation becomes more structured and slightly more complex, the resulting system is more robust, organised, and adaptable to future requirements compared to the original design.

Limitations of the final design and implementation

Despite the architectural improvements introduced through the application of design patterns, the final design and implementation of the Flight Booking System still present several limitations. The use of the Singleton Pattern in the DatabaseConnection class centralises database access effectively, but it also creates a globally shared dependency that reduces flexibility for dependency injection and automated unit testing. Repository classes remain directly dependent on DatabaseConnection.Instance, which may complicate future testing and system refactoring.

The Factory Method implementation also introduces some scalability limitations. While the UserFactory class successfully centralises the creation of Customer and Admin objects, the factory must still be modified whenever additional user types are introduced. As the system expands, this may increase the complexity of the factory implementation. Additionally, the booking workflow remains simplified compared to real-world airline reservation systems. Bookings are immediately confirmed

without payment verification, reservation holding, or advanced booking constraints such as seat selection and concurrent booking management. The system is also limited by its console-based interface and strong dependency on Oracle-specific database technologies, which reduce usability and database portability. Although the refined design significantly improves maintainability and separation of concerns, these limitations highlight areas for future architectural and functional enhancement.

Lessons learned from aligning UML models with C# code.
Aligning the UML models with the C# implementation highlighted the importance of keeping design documentation consistent with the actual system code. Through this process, I learned that UML diagrams should not only represent the intended design, but also accurately reflect how classes, methods, relationships, and responsibilities are implemented in the codebase. For example, changes such as making User a superclass, adding Customer and Admin subclasses, and introducing the UserFactory required both the UML class diagram and the C# code to be updated together.
This alignment also helped identify inconsistencies between the design and implementation, such as whether the system was role-based or inheritance-based, and whether PENDING should appear in the booking state machine if bookings are created directly as CONFIRMED. As a result, the design became clearer, more accurate, and easier to justify. Overall, the process showed that UML is not just a planning tool, but also a way to validate, refine, and communicate the structure of the implemented system.